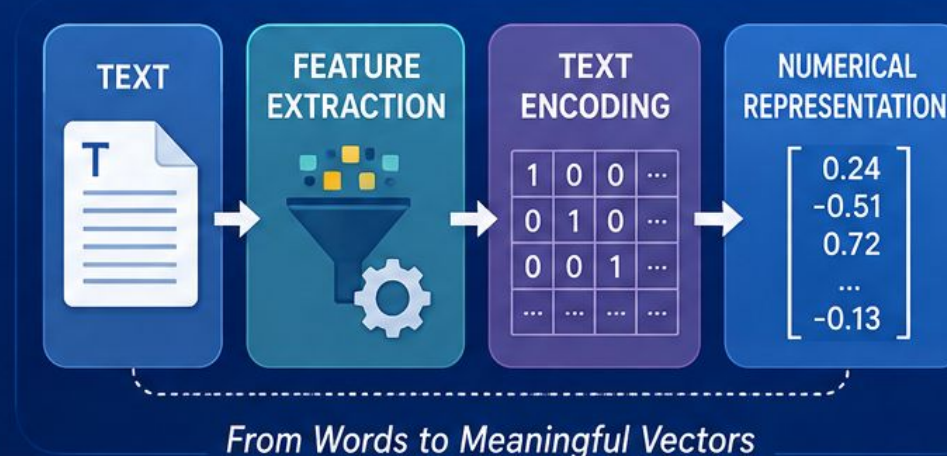




WEEK 3

FEATURE EXTRACTION & TEXT ENCODING

How we convert text into numerical representations that machines can understand and learn from.



IN THIS LECTURE

- ✓ Understand different ways to represent text numerically
- ✓ Learn encoding techniques: Integer, One-Hot, Multi-Hot
- ✓ Explore advantages and limitations of traditional encodings
- ✓ Bridge to Word Embeddings and Word2Vec (CBOW & Skip-Gram)



Duration
2 HOURS



Level
MSc AI
Level 2



Focus
Representation
& Encoding



Outcome
Text → Numbers →
Meaningful Vectors



LEARNING OUTCOMES

By the end of this lecture, you should be able to:

1



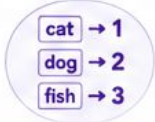
Define and explain feature extraction

2



Build and use vocabularies

3



Apply integer (label) encoding

4



Implement one-hot encoding

5



Compare basic text representations

6



Explain the limitations of sparse representations

7



Describe dense word embeddings

8



Understand the distributional hypothesis

9



Explain Word2Vec architectures
(CBOW & Skip-Gram)

10



Connect traditional encodings to Word2Vec



KEY TAKEAWAY

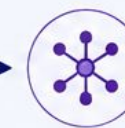
We start with basic encodings (integers, one-hot, counts) and progress toward dense word embeddings learned by Word2Vec (CBOW & Skip-Gram) that capture meaning and context.



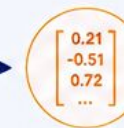
Text



Encoding



Word2Vec



Meaningful
Vectors

WHY COMPUTERS NEED NUMBERS



Computers can only understand **numbers**.
To process text, we must convert words into numerical representations that machines can **store**, **compare** and **learn** from.



ANALOGY
Humans read letters and words.
Computers read numbers.
We translate text into numbers so computers can make sense of it.

WHY NUMBERS MATTER IN NLP



Computers do math, not language
Machines perform calculations. Numbers allow them to measure, compare and compute.



ML/AI models learn from numbers
Algorithms (e.g., neural networks) learn patterns from numerical data, not words.



Consistency and efficiency
Numbers use less memory, are faster to process and easy to manipulate.



Capture meaning in a structured way
Numerical representations (embeddings, vectors) capture semantic relationships between words.

EXAMPLES: FROM WORDS TO NUMBERS

1. Simple Mapping (e.g., Label Encoding)

Word	cat	dog	bird
Number	1	2	3

2. One-Hot Encoding

Word	cat	dog	bird
cat	1	0	0
dog	0	1	0
bird	0	0	1

3. Word Embedding (Vector Representation)

Word	cat	[0.21, -0.35, 0.12, ...]
	dog	[0.18, -0.30, 0.08, ...]
	bird	[-0.12, 0.45, -0.22, ...]



Modern NLP uses dense vector **embeddings** that capture meaning and context, not just simple IDs.

WHAT WE CAN DO WITH NUMBERS



Measure similarity between words (e.g., king and queen are close).



Train models to classify, predict or generate text.



Understand context, sentiment and intent.



Index and retrieve information efficiently.



Visualize data and build insights.

★ KEY TAKEAWAY

Text must be converted into numbers so that computers can understand, learn from, and reason over language. This is the **foundation of all NLP systems!**



REMEMBER
No numbers → No computation.
No computation → No intelligence.

WHAT IS FEATURE EXTRACTION?

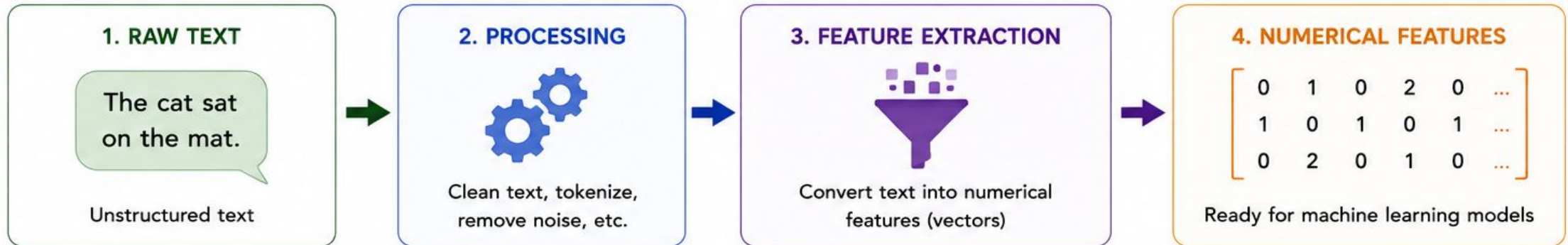


Feature extraction is the process of **converting raw text** into a set of meaningful **numerical features** (vectors) that a machine learning model can understand and use.

WHY IS IT IMPORTANT?

- ✓ Computers can only work with **numbers**.
- ✓ Features capture the most **useful information** from text.
- ✓ Good features **improve model performance**.
- ✓ It **reduces complexity and noise** in the data.

THE FEATURE EXTRACTION PIPELINE



EXAMPLES OF FEATURES


Word counts
(Bag of Words)

$\frac{tf}{idf}$
TF-IDF
weights

the	cat
cat	sat

N-grams
(bi-grams, tri-grams)

10101
01010
One-hot
vectors


Frequency
distributions


POS tags,
sentence length, etc.



KEY TAKEAWAY

Feature extraction transforms text into numerical representations that preserve important information and enable machines to learn patterns.

WHY DO WE NEED FEATURE EXTRACTION?



Raw text is unstructured and complex. Feature extraction transforms it into **useful numerical information** that machines can process, learn from, and make decisions with.

THE PROBLEM WITH RAW TEXT



The cat sat
on the mat.

- ✗ Unstructured and variable length
- ✗ High dimensional and sparse
- ✗ Contains noise and irrelevant words
- ✗ Hard for computers to understand
- ✗ Cannot be used directly in ML models

WE NEED FEATURE EXTRACTION TO:



Convert text into numbers

Machines only understand numerical data.



Capture the important information

Represents the most relevant patterns and meanings.



Improve model performance

Better features lead to more accurate predictions.



Reduce dimensionality and noise

Removes irrelevant data and focuses on what matters.



Make learning efficient

Simplifies data and speeds up training.



Enable generalization

Helps models understand and apply to new data.

WITHOUT vs WITH FEATURE EXTRACTION

WITHOUT FEATURE EXTRACTION



- Computers get confused
- Low accuracy
- Slow and inefficient
- Poor generalization

VS

WITH FEATURE EXTRACTION



- Computers understand
- High accuracy
- Faster and efficient
- Better generalization

EXAMPLE

The cat
sat on the mat.



Process
(clean, tokenize)



Extract Features
(counts, n-grams, etc.)



Numerical Features
(vector representation)



Machine Learning
Model

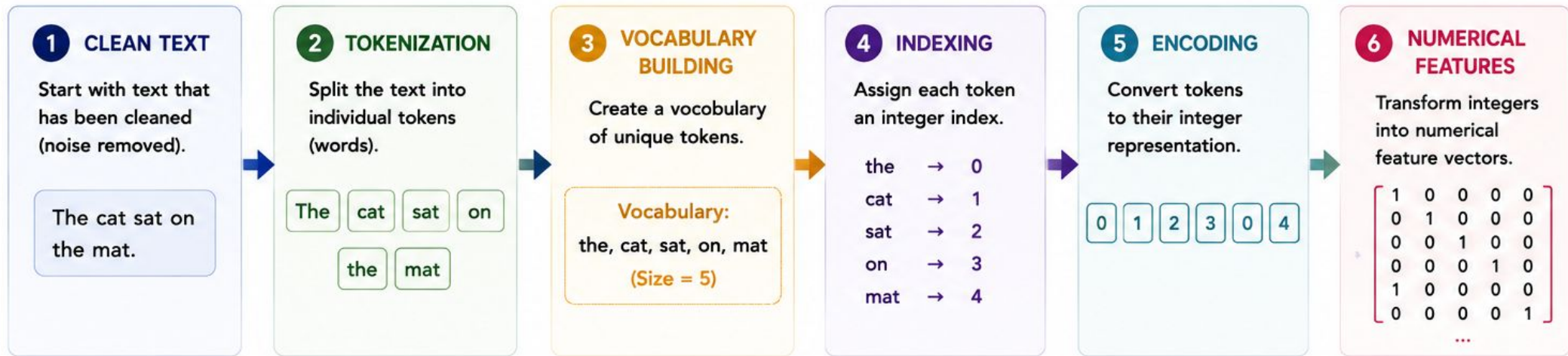


KEY TAKEAWAY

Feature extraction is the bridge between raw text and intelligent models. It turns unstructured text into clear, compact, and meaningful representations that drive the success of NLP systems.

FROM CLEAN TEXT TO NUMERICAL FEATURES

The goal is to convert cleaned, meaningful text into **numbers (vectors)** that machine learning models can understand and learn from.



EXAMPLE Sentence: *The cat sat on the mat.*

Clean Text	Tokens	Vocabulary	Indices (Mapping)	Integer Sequence	One-Hot Feature Matrix
The cat sat on the mat.	The cat sat on the mat	0: the 1: cat 2: sat 3: on 4: mat	the → 0 cat → 1 sat → 2 on → 3 mat → 4	0 1 2 3 0 4 (Length = 6)	$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$ (6 words × 5 features)



KEY TAKEAWAY

Clean text is progressively converted through a series of steps into **numerical feature vectors** that capture the information in a format usable by machine learning models.



From raw words to numerical vectors – this transformation is the foundation of all NLP models.

WHAT IS A FEATURE?



A feature is a **measurable property** or **characteristic** extracted from text that **helps a machine learning model understand, learn patterns, and make predictions.**

KEY IDEAS

-  Features are the **input** to a machine learning model.
-  They represent **important information** about the text.
-  Better features lead to **better model performance.**
-  Features convert unstructured text into **structured data.**

EXAMPLES OF FEATURES IN TEXT PROCESSING

WORD COUNT

Number of words in the text.

Example

The cat sat on the mat.

Count = 6

TERM FREQUENCY (TF)

How many times a term appears in the text.

Example

Term	TF
cat	1
sat	1
mat	1
the	2

N-GRAMS

Sequences of N consecutive words.

Example (bi-grams)

The cat
cat sat
sat on
on the
the mat

PART-OF-SPEECH TAGS

Grammatical category of each word.

Example

The/DET cat/NOUN
sat/VERB on/ADP
the/DET mat/NOUN
./PUNCT

SENTIMENT SCORE

Polarity or emotion of the text.

Example

Text: "I love this product!"
Sentiment Score = +0.85

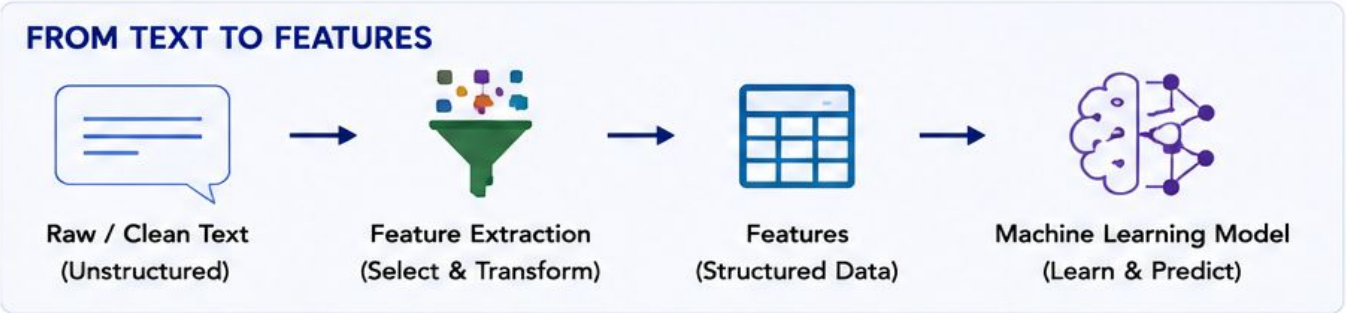
TEXT LENGTH

Number of characters or tokens.

Example

Characters = 23
Tokens = 6

FROM TEXT TO FEATURES



KEY TAKEAWAY

A feature is any meaningful information extracted from text that can be represented as a number. Good features capture the essence of the text and enable models to learn effectively.

TEXT FEATURES vs NUMERICAL FEATURES

Machine learning models cannot understand **raw text** directly.
Text features must be converted into **numerical features**.

TEXT FEATURES



Also called **unstructured features**.
They are in the form of words, phrases, sentences or documents.

Nature	Unstructured, human readable
Examples	Words, sentences, paragraphs, documents, reviews, emails, tweets
What do they represent?	Raw information in natural language
Can ML models use them?	No, they must be transformed into numerical features
Example	<i>"The cat sat on the mat."</i>

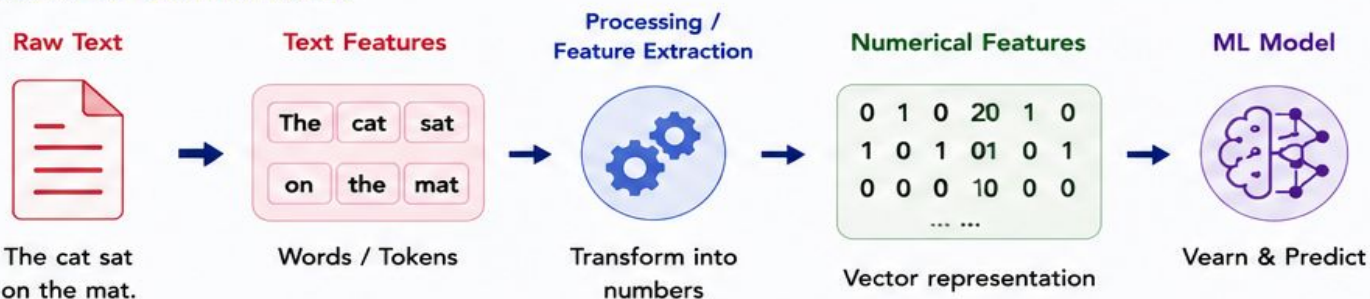
NUMERICAL FEATURES



Also called **structured features**.
They are numbers (or vectors of numbers) that machines can process.

Nature	Structured, machine readable
Examples	Counts, frequencies, binary indicators, embeddings, scores, lengths
What do they represent?	Quantitative representation of information
Can ML models use them?	Yes, directly used as input to ML models
Example	[0 1 2 0 3 1 0 4 ...]

EXAMPLE COMPARISON



KEY TAKEAWAY

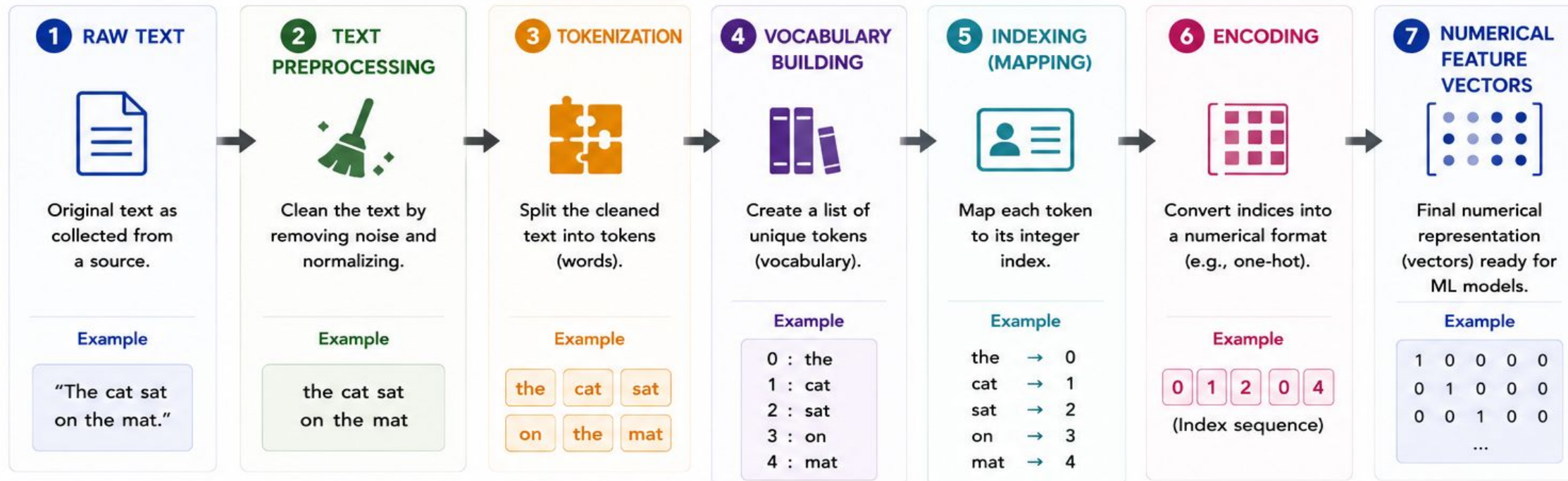
- **Text features** are human language.
- **Numerical features** are numbers that represent the information.
- **Feature extraction** is the bridge between text and machine learning.

THE TEXT REPRESENTATION PIPELINE



BIG PICTURE

We convert messy, unstructured text into clean, meaningful, numerical representations.



WHAT HAPPENS IN EACH STEP?

- Raw text can be noisy, inconsistent, and unstructured.
- Example: text from reviews, tweets, articles, etc.
- Lowercasing
- Remove punctuation
- Remove stopwords
- Remove extra spaces
- Stemming / lemmatization
- Split by spaces
- Handle punctuation and contractions
- Result: list of tokens (words)
- Collect all unique tokens
- Assign an integer index to each token
- Size = vocabulary size ($|V|$)
- Replace each token with its integer index
- Creates a sequence of numbers
- Integer sequence may be converted to one-hot, count vectors, or other encoding schemes
- Dense or sparse vectors represent the text
- These are used as input to machine learning models



KEY TAKEAWAY: The pipeline turns unstructured text into structured numerical features that capture information in a format machines can learn from.

WHAT IS TEXT ENCODING?



Text encoding is the process of converting text (tokens or indices) into **numerical format (vectors)** so that **machine learning models** can process, learn from, and make predictions.

WHY ENCODE TEXT?



Computers understand numbers, not text
Models operate on numerical data only.



Capture meaning in a numeric form
Encodings represent relationships between words.



Enable machine learning
Essential for training and making accurate predictions.



Handle variable-length text
Convert any length of text into fixed-size vectors.

EXAMPLE ILLUSTRATION

1. Text

"The cat sat on the mat."

2. Tokens

the cat sat on the mat

3. Indices

0 1 2 3 0 4

4. Encoding

One-Hot Encoding (size = 5)

the (0)	1	0	0	0	0
cat (1)	0	1	0	0	0
sat (2)	0	0	1	0	0
on (3)	0	0	0	1	0
mat (4)	0	0	0	0	1

(6 words × 5 features)

THE BIG PICTURE PIPELINE



Raw Text
(Unstructured)



Preprocessing
(Clean Text)



Tokenization
(Words / Tokens)



Indexing
(Numbers)



Encoding
(Vectors)



Machine Learning Model
(Learn & Predict)

COMMON ENCODING METHODS



One-Hot Encoding

Each word is represented by a binary vector with 1 at its index and 0 elsewhere.
Simple but high-dimensional.



Count / Frequency Encoding

Words are represented by their frequency counts in the text.



TF-IDF Encoding

Weights words based on how important they are in a document compared to the whole corpus.



Word Embeddings (e.g., Word2Vec, GloVe, BERT)

Words are mapped to dense, low-dimensional vectors that capture semantic meaning and relationships.



KEY TAKEAWAY

- Text encoding turns words into numbers.
- It is the bridge between natural language and machine learning.
- The choice of encoding affects model performance and efficiency.

WHY CAN'T WE FEED WORDS DIRECTLY TO A MACHINE LEARNING MODEL?

Machine learning models operate on **NUMBERS**, not **WORDS**. Raw text must be converted into numerical representations that capture the meaning and patterns in the data.



1. MODELS REQUIRE NUMBERS

Most ML algorithms (e.g., linear regression, neural networks) perform mathematical operations such as addition, multiplication, and dot products. These operations are not defined on text strings.

Example:

"cat" + "dog" ❌ (not valid)

0.25 + 0.75 ✅ (valid)



2. WORDS HAVE NO NATURAL NUMERICAL MEANING

Words are symbols. They do not have an inherent magnitude or order that models can exploit. For example, "cat" is not numerically closer to "dog" than to "car".

Example:

cat	dog	car
No numerical distance exists between these words.		



3. DIMENSIONALITY MISMATCH

The vocabulary of a language can be extremely large (thousands or millions of unique words), while most ML models expect input vectors of fixed, much smaller length.

Example:

10,000 unique words →
Model expects vectors of size 128
(huge mismatch)



4. LACK OF GENERALIZATION

If a model sees a new word during inference that was not seen during training, it cannot reason about it. This leads to poor generalization.

Example:

Training: "cat", "dog"

Inference: "hamster"

Model has no prior knowledge of "hamster".

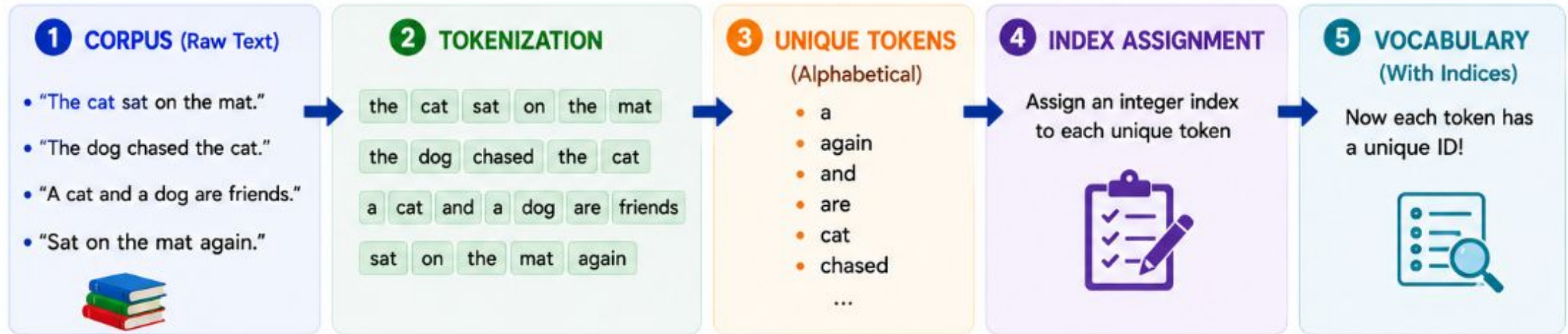


KEY TAKEAWAY

Text must be transformed into numerical features that capture the semantic and syntactic information of the words. This is the role of text encoding and feature extraction techniques.

BUILDING A VOCABULARY — WORKED EXAMPLE

A vocabulary is the list of all **unique tokens (words)** in your corpus.
Each token will later be assigned an **index** for encoding.



RESULTING VOCABULARY			
Index	Token	Index	Token
0	a	6	dog
1	again	7	friends
2	and	8	mat
3	are	9	on
4	cat	10	sat
5	chased	11	the

ENCODED REPRESENTATION (Using Indices)	
Sentence	Indexed Sequence (token IDs)
"The cat sat on the mat."	[11, 4, 10, 9, 11, 8]
The dog chased the cat."	[11, 6, 5, 11, 4]
A cat and a dog are friends.	[0, 4, 2, 0, 6, 3, 7]
Sat on the mat again."	[10, 9, 11, 8, 1]

KEY TAKEAWAYS

- ✓ Vocabulary = all unique tokens in the corpus.
- ✓ Each token gets a unique index.
- ✓ These indices are used for encoding.
- ✓ Smaller vocabulary → simpler models.
- ✓ Large vocabulary → more expressive models (but more memory).



Remember: The order of indices is arbitrary but must be consistent throughout your model.

INTEGER / LABEL ENCODING



Integer (or Label) Encoding converts each unique token (word or category) into a unique integer (label). It is simple, fast, and commonly used as a first step before other representations.

HOW IT WORKS



- 1 Build Vocabulary**
Collect all unique tokens from the corpus and assign each a unique integer (typically starting from 0).



- 2 Create Mapping**
Create a mapping (token → integer) and optionally the reverse (integer → token).



- 3 Transform Text**
Replace each token in the text with its corresponding integer.



ADVANTAGES

- Simple to implement
- Memory efficient
- Works well for tree-based models
- Good baseline representation



LIMITATIONS

- Implies ordinal relationship (which does not exist for words)
- Not suitable for distance-based models (e.g., linear models, k-NN)
- Loses semantic relationships

WORKED EXAMPLE

Example corpus (after preprocessing):

Doc 1: the cat sat on the mat
Doc 2: the dog sat on the log
Doc 3: the cat chased the dog

1) VOCABULARY (Unique tokens)

the	cat	sat	on	mat	dog	log	chased
-----	-----	-----	----	-----	-----	-----	--------

2) INTEGER / LABEL MAPPING

Token	the	cat	sat	on	mat	dog	log	chased
Integer ID	0	1	2	3	4	5	6	7

Unique integer assigned to each token

3) TRANSFORM TEXT (Tokens → Integers)

Document	Original (Tokens)						Encoded (Integers)					
Doc 1	the	cat	sat	on	the	mat	0	1	2	3	0	4
Doc 2	the	dog	sat	on	the	log	0	5	2	3	0	6
Doc 3	the	cat	chased	the	dog		0	1	7	0		5



Key Takeaway: Integer/Label Encoding assigns a unique number to each token. It is easy and efficient, but the numbers do not carry meaning or similarity.



WHEN TO USE

- As a baseline representation
- For tree-based models (Decision Trees, Random Forests, XGBoost, LightGBM)
- When combined with Embeddings (as input to embedding layers)

QUICK EXAMPLE

For the word “cat”

- Integer ID = 1
- Reverse mapping: 1 → “cat”

Integer ID	Token
0	the
1	cat
...	...
7	chased



IMPORTANT NOTES

- Integer IDs are arbitrary. They do not imply order.
- Do not use integer encoding with linear or distance-based models without embeddings.
- Always keep the mapping for decoding and interpretation.

PROBLEMS WITH INTEGER ENCODING



Integer encoding assigns arbitrary numbers to words.

Models may wrongly treat these numbers as having **order or distance**, which is not true for words.

1 IMPLIES FALSE ORDER

Numbers create an artificial order between words that does not exist.

Example

Word	ID
dog	2
cat	5
tiger	9

Model thinks "tiger" is greater than "dog"!



But words have no natural order.

2 IMPLIES FALSE DISTANCE

Numeric distance between IDs does not reflect semantic similarity.

Example

Word	ID
cat	5
dog	2
tiger	9

$$|5 - 2| = 3$$

$$|5 - 9| = 4$$

Model thinks cat is closer to dog than to tiger.



Gaps between IDs mean nothing for words.

3 POOR FOR SEMANTIC GENERALIZATION

IDs do not capture meaning or relationships. Unseen words cannot be inferred from numbers.

Example

Model sees:

king = 1, queen = 2

What about prince?

ID 3 gives no clue!



No information about meaning, gender, royalty, etc.

4 NOT SUITABLE FOR NEURAL MODELS ALONE

Neural networks need dense, distributed representations (embeddings) to learn rich patterns.

Example

Feeding IDs directly:

0	5	11	4	9	11	8
---	---	----	---	---	----	---



Neural Model

Sparse, arbitrary inputs limit learning.



Use embeddings (Word2Vec, GloVe, BERT) for neural models.

5 SENSITIVE TO VOCABULARY CHANGES

Adding or removing words changes the IDs of many words, breaking consistency across datasets or over time.

Example

Old mapping: dog = 2, cat = 5

Add new word "apple" at start

New mapping: apple = 0,

dog = 3, cat = 6

Word	Old ID	New ID
apple	—	0
dog	2	3
cat	5	6



Models trained earlier may behave inconsistently.



KEY TAKEAWAY

Integer encoding is simple and works for basic models (e.g., Decision Trees).

But it introduces artificial order and distances, so use embeddings or one-hot encoding for better results—especially with neural models.

WHAT IS ONE-HOT ENCODING?



One-Hot Encoding is a way to represent each unique word (or category) as a **binary vector**.

Each word gets its own column.

The word is represented by a **1** in its column and **0** in all other columns.

Key Idea:

It removes any notion of **order** or **distance** between words.

The model sees only presence (**1**) or absence (**0**) of a word.

SIMPLE ANALOGY



Think of a **class attendance list**.

If there are 4 students (A, B, C, D), you mark '1' if the student is present and '0' if absent.

	A	B	C	D
Student A	1	0	0	0
Student B	0	1	0	0
Student C	0	0	1	0
Student D	0	0	0	1

WHY USE ONE-HOT ENCODING?



- It treats all words equally.
- No artificial order or distance.
- Works well with linear models and neural networks.
- It is simple, safe, and effective.

HOW IT WORKS – EXAMPLE

Suppose our vocabulary is: [the, cat, sat, on, mat]

Vocabulary size = 5 (so each word will be represented by a vector of length 5)

Word	the	cat	sat	on	mat
the	1	0	0	0	0
cat	0	1	0	0	0
sat	0	0	1	0	0
on	0	0	0	1	0
mat	0	0	0	0	1

ENCODING A SENTENCE

Sentence: "the cat sat on the mat"

Words	the	cat	sat	on	the	mat
						
One-Hot Vectors	1 0 0 0 0	0 1 0 0 0	0 0 1 0 0	0 0 0 1 0	1 0 0 0 0	0 0 0 0 1
						
(5 × 6 matrix)						

BENEFITS



- No order or ranking between words.
- No distance between words.
- Each word is independent.
- Helps models learn better patterns.



KEY TAKEAWAY

One-Hot Encoding turns words into vectors of 0s and 1s where only one position is 1.

It removes any false notion of order or distance and is a strong representation for machine learning models.

0	0	0	1	0
---	---	---	---	---

(One-Hot Vector Example)

WHY NUMBERS CAN ACCIDENTALLY IMPLY ORDER



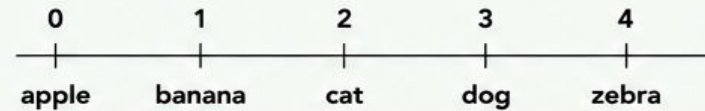
When we use integers to represent words, models that use numbers (especially linear models, distance-based methods, or algorithms that assume order) treat higher numbers as “greater” and lower numbers as “smaller”. But word IDs are just labels — they **do not have any natural order!**

EXAMPLE VOCABULARY (Integer IDs)

apple	banana	cat	dog	zebra
0	1	2	3	4

HOW A MODEL MAY INTERPRET THESE NUMBERS

The model sees a numeric scale:



Model assumes:
apple < banana < cat < dog < zebra
(an order)

WHAT THIS MEANS (AND WHY IT'S A PROBLEM)



1. IMPLIES AN ORDER THAT DOES NOT EXIST

The model thinks:
“dog” is greater than “cat”
“zebra” is much greater than “apple”

But in reality, these words have **no natural ranking**.



2. IMPLIES DISTANCE BETWEEN WORDS

The model thinks the distance between IDs has meaning.
For example:
 $|\text{dog (3)} - \text{cat (2)}| = 1$
 $|\text{zebra (4)} - \text{apple (0)}| = 4$

But these distances mean nothing semantically.



3. CAN MISLEAD THE MODEL

The model may learn patterns based on these fake orders and distances, leading to **incorrect predictions** or poor generalization.

4. ESPECIALLY HARMFUL FOR LINEAR / DISTANCE-BASED MODELS



Algorithms that rely on magnitude or distance (e.g., linear regression, k-NN, SVM with linear kernel) will be **misled**.

A CONCRETE EXAMPLE

Suppose a model learns that **larger numbers** are associated with **positive labels**.

If the model learns: **higher ID → better outcome**, it may predict “zebra” (ID=4) is better than “apple” (ID=0) — which is nonsense!

Input word
ID = 4 (zebra)



Model thinks
4 is “larger”



Model predicts
Positive / “Better”



But in reality, zebra is **not** necessarily **“better”** than apple!



KEY TAKEAWAY

Integer IDs are **arbitrary labels**, not meaningful numbers.
Models can wrongly use the numeric order and distance, leading to poor or misleading results.
That’s why we need representations (like one-hot encoding or embeddings) that **do not imply order**.



Goal: Use representations that capture **meaning**, not arbitrary numbers.

CREATING A ONE-HOT VECTOR — WORKED EXAMPLE



One-Hot Encoding represents each word as a **binary vector of 0s and 1s**.

The word gets a **1** in its own position and **0** in all other positions.

1 BUILD THE VOCABULARY



List all unique words in the corpus.

2 ASSIGN POSITIONS

1 2 3 4 5

Give each word a unique position (index) from 1 to V (vocab size = 5).

3 CREATE ONE-HOT VECTOR FOR EACH WORD

0 0 1 0 0

Put 1 in its position and 0 in all other positions.

4 ENCODE THE SENTENCE



Replace each word in the sentence with its one-hot vector.

5 USE THE MATRIX AS MODEL INPUT



The resulting matrix of 0s and 1s is fed to the model.

VOCABULARY (V = 5)

Word	Index / Position	One-Hot Vector
the	1	[1, 0, 0, 0, 0]
cat	2	[0, 1, 0, 0, 0]
sat	3	[0, 0, 1, 0, 0]
on	4	[0, 0, 0, 1, 0]
mat	5	[0, 0, 0, 0, 1]

ENCODE THE SENTENCE

Sentence: "the cat sat on the mat"

Step 1: Split into words

the cat sat on the mat

Step 2: Replace each word with its one-hot vector

$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$

Resulting matrix (V × N) = (5 × 6)

FINAL ONE-HOT MATRIX

Words → Index ↓	the	cat	sat	on	the	mat
1 (the)	1	0	0	0	1	0
2 (cat)	0	1	0	0	0	0
3 (sat)	0	0	1	0	0	0
4 (on)	0	0	0	1	0	0
5 (mat)	0	0	0	0	0	1

- Rows = vocabulary words (5)
- Columns = words in the sentence (6)



WHAT WE ACHIEVED

- Each word is represented by a unique binary vector.
- There is no implied order or distance between words.
- The model sees only the presence (1) or absence (0) of a word.

ONE-HOT VECTOR EXAMPLE

Word = "sat" → [0, 0, 1, 0, 0]
(Position 3 = 1)

KEY FORMULA

If vocabulary size = V
and sentence length = N
Then one-hot matrix size = V × N

ONE-HOT ENCODING A SENTENCE



One-Hot Encoding converts each word in a sentence into a **binary vector** of **0s** and **1s**. Each word gets a **1** in its own position and **0** in all other positions.

1 BUILD THE VOCABULARY



List all unique words in the corpus / sentence.

2 ASSIGN POSITIONS

1 2 3 4 5

Give each word a unique position (index) from 1 to V (vocabulary size).

3 CREATE ONE-HOT VECTOR FOR EACH WORD

0 0 1 0 0

Put 1 in its position and 0 in all other positions.

4 ENCODE THE SENTENCE



Replace each word in the sentence with its one-hot vector.

5 FORM THE MATRIX AND USE AS INPUT



The matrix of 0s and 1s is fed to the model.

1 VOCABULARY AND POSITIONS

Word	Index / Position	One-Hot Vector
the	1	[1, 0, 0, 0, 0, 0]
cat	2	[0, 1, 0, 0, 0, 0]
sat	3	[0, 0, 1, 0, 0, 0]
on	4	[0, 0, 0, 1, 0, 0]
mat	5	[0, 0, 0, 0, 1, 0]
again	6	[0, 0, 0, 0, 0, 1]

Vocabulary size (V) = 6

2 THE GIVEN SENTENCE

"the cat sat on the mat again"

Step 1: Split into words

the cat sat on the mat again

Step 2: Map each word to its index

1 2 3 4 1 5 6



Both occurrences of "the" use the same index (1).

3 ONE-HOT VECTORS FOR EACH WORD

the	cat	sat	on	the	mat	again
↓	↓	↓	↓	↓	↓	↓
1	0	0	0	1	0	0
0	1	0	0	0	0	0
0	0	1	0	0	0	0
0	0	0	1	0	0	0
0	0	0	0	0	1	0
0	0	0	0	0	0	1



Each column is the one-hot vector of the corresponding word.

4 FINAL ONE-HOT MATRIX (V × N)

Rows → words (V = 6)

Columns → words in sentence (N = 7)

	1	2	3	4	5	6	7
the (1)	1	0	0	0	1	0	0
cat (2)	0	1	0	0	1	0	0
sat (3)	0	0	1	0	0	0	0
on (4)	0	0	0	1	0	0	0
mat (5)	0	0	0	0	0	0	0
again (6)	0	0	0	0	0	0	1

Resulting shape: 6 × 7 matrix
(Vocabulary size × Sentence length)



KEY TAKEAWAYS

- ✓ One-Hot Encoding creates a binary vector for each word.
- ✓ It removes any notion of order or distance between words.
- ✓ The model sees only presence (1) or absence (0) of a word.
- ✓ Each column in the matrix represents one word in the sentence.

ONE-HOT VECTOR EXAMPLE

Word = "sat" (index 3)

[0, 0, 1, 0, 0, 0]

(Length = 6)

NOTES

- V = vocabulary size (number of unique words)
- N = sentence length (number of words)
- Matrix size = V × N
- This matrix is typically used as input to machine learning models (e.g., linear models, neural networks).



ONE-HOT ENCODING IN PYTHON



One-Hot Encoding converts categorical data into **binary** columns (**0s** and **1s**). Each category becomes a column with **1** (present) or **0** (absent).



WHY USE ONE-HOT ENCODING?

- ✓ Removes any notion of order or distance
- ✓ Treats all categories equally
- ✓ Works well with linear models and neural networks
- ✓ Simple, safe, and effective

1 IMPORT LIBRARIES

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
```

2 CREATE SAMPLE DATA

```
# Sample dataset
data = pd.DataFrame({
    'Animal': ['dog', 'cat', 'dog', 'rabbit', 'cat']
})

print("Original Data:")
print(data)
```

Original Data:

	Animal
0	dog
1	cat
2	dog
3	rabbit
4	cat

3 APPLY ONE-HOT ENCODING

```
# Create OneHotEncoder object
encoder = OneHotEncoder(sparse_output=False)

# Fit and transform the data
encoded_array = encoder.fit_transform(data[['Animal']])

# Get the encoded column names
encoded_columns = encoder.get_feature_names_out(['Animal'])

# Convert to DataFrame
encoded_df = pd.DataFrame(encoded_array,
                          columns=encoded_columns)

print("One-Hot Encoded Data:")
print(encoded_df)
```

5 ONE-HOT ENCODED RESULT

One-Hot Encoded Data:

	Animal_cat	Animal_dog	Animal_rabbit
0	0.0	1.0	0.0
1	1.0	0.0	0.0
2	0.0	1.0	0.0
3	0.0	0.0	1.0
4	1.0	0.0	0.0

4 COMBINE WITH ORIGINAL DATA (OPTIONAL)

```
result = pd.concat([data, encoded_df], axis=1)

print("Final Data (Original + One-Hot Encoded):")
print(result)
```

Final Data (Original + One-Hot Encoded):

	Animal	Animal_cat	Animal_dog	Animal_rabbit
0	dog	0.0	1.0	0.0
1	cat	1.0	0.0	0.0
2	dog	0.0	1.0	0.0
3	rabbit	0.0	0.0	1.0
4	cat	1.0	0.0	0.0

TWO WAYS TO DO IT IN PYTHON

1. Using pandas.get_dummies()

```
dummies = pd.get_dummies(
    data, columns=['Animal'])

print(dummies)
```

2. Using sklearn OneHotEncoder (shown above)

- More control
- Works well in ML pipelines



KEY TAKEAWAYS

- One-Hot Encoding creates a binary column for each category.
- Only one column has 1 for a given row; others are 0.
- It removes any artificial order or distance between categories.
- The resulting matrix can be used directly as input to ML models.



NOTE

- If your data has many categories, the number of columns (features) increases.
- In such cases, consider techniques like frequency encoding or target encoding.



ADVANTAGES OF ONE-HOT ENCODING



One-Hot Encoding represents each category as a **binary vector** (0s and 1s). This simple representation brings several important advantages.



1. Eliminates False Ordinal Relationships

- One-hot encoding removes any artificial **order** or **ranking** between categories.
- The model does not assume that “dog (2)” is closer to “cat (1)” than to “tiger (9)”.



2. Treats All Categories Equally

- Each category is given the **same type** of representation.
- This prevents bias toward categories with smaller or larger integer values.



3. Works Well with Most Machine Learning Models

- Many algorithms (e.g., linear models, logistic regression, neural networks) **work better** with numeric inputs that are not ordinal.
- One-hot encoding is widely compatible and model-friendly.



4. Improves Model Interpretability

- Each column represents one **specific category**.
- It becomes easy to understand how much each category contributes to the prediction.



5. Simple, Safe and Effective

- Very easy to implement.
- Introduces **no misleading information**.
- Provides a clean and robust representation for categorical data.



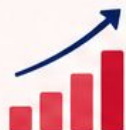
KEY TAKEAWAY

One-Hot Encoding converts categories into a **binary**, **non-ordinal**, and **equal** representation. This helps machine learning models focus on **real patterns**, not artificial numbers.

LIMITATIONS OF ONE-HOT ENCODING



One-Hot Encoding is simple and powerful, but it also has some important **limitations**. Understanding these helps us choose the right encoding technique for the right problem.

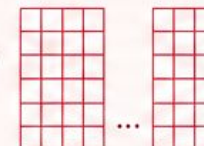


1. High Dimensionality (Sparsity)

- Creates **as many columns** as the number of categories.
- Leads to very wide and **sparse** feature matrices.
- Can increase memory usage and computation time.

Example:

If a feature has 10,000 categories, it will create 10,000 columns. Most values will be 0.



2. Not Scalable to High Cardinality

- With **very high-cardinality** features (e.g., user_id, product_id), one-hot encoding becomes impractical.
- Can lead to performance issues in training and storage.

Example:

Encoding user_id of millions of users results in millions of columns. Not scalable!

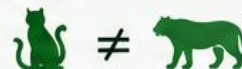


3. Ignores Relationships Between Categories

- One-hot encoding treats all categories as completely **independent**.
- It cannot capture any **similarity** or hierarchy between categories.

Example:

It treats "Cat" and "Tiger" as unrelated, even though they are similar.



4. More Computational Cost

- More features = more computations.
- Increases training time and prediction time, especially for large datasets.

Example:

Training a model with 50,000 one-hot features takes much longer than with fewer features.



5. Not Suitable for Tree-Based Models (Sometimes)

- Tree-based algorithms (like Decision Trees, Random Forests) can handle integer encoding well. One-hot encoding may add unnecessary complexity.

Example:

For tree models, label/integer encoding is often sufficient and more efficient.



KEY TAKEAWAY

One-Hot Encoding is excellent for low to medium cardinality categorical features and works very well with linear models and neural networks. For high-cardinality features, consider alternatives like **Frequency Encoding**, **Target Encoding**, **Hashing Encoding**, or **Embeddings**.



THE SPARSE VECTOR PROBLEM



One-Hot Encoding creates vectors that are mostly **0s** with only a single **1**.
For large vocabularies or many categories, this leads to very **sparse vectors**.

EXAMPLE: LARGE VOCABULARY

Vocabulary size (V) = 10,000 words

One word:
"elephant"

One-hot vector length = 10,000



9,999 zeros

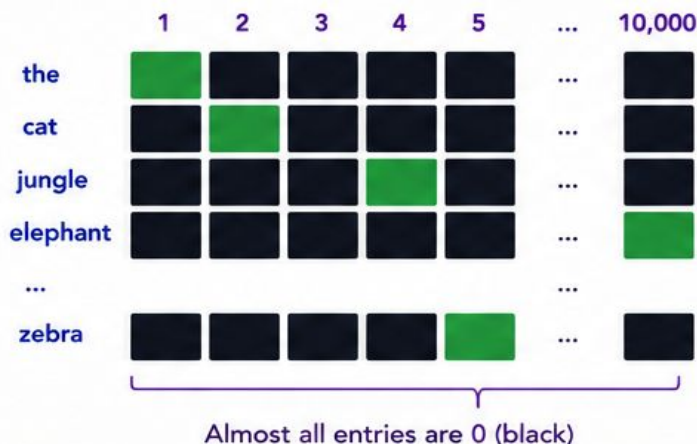
Only one 1



99.99% of the entries are **0**,
Only **0.01%** carries information!

Visualizing Sparsity

Each row represents one word.
Each column is one word in the vocabulary.
Black = 0, Green = 1



WHY IS THIS A PROBLEM?



1. High Memory Usage

Storing many sparse vectors takes a lot of memory.



2. More Computation

Operations (multiplication, addition) waste time on zeros.



3. Inefficient Storage

Most storage is used to store zeros, which carry no information.



4. Not Scalable

As vocabulary/categories grow, vectors become extremely large and hard to handle.

REAL-WORLD CASE



Example: If a model uses a vocabulary of **100,000** words, each word is represented by a vector of length **100,000** with only **ONE** non-zero value.



- Storing 1 million such vectors → **100 billion entries!**
- But only **1 million** of them are 1s.
- 99.999%** are zeros.



This is called the **SPARSE VECTOR PROBLEM**.



KEY TAKEAWAY

One-Hot Encoding creates extremely **sparse vectors** — mostly **0s** with only one **1**.
This leads to high memory usage, more computation, and poor scalability for large vocabularies.
To handle this, we use alternative encoding techniques like:

Frequency Encoding

Target Encoding

Hashing

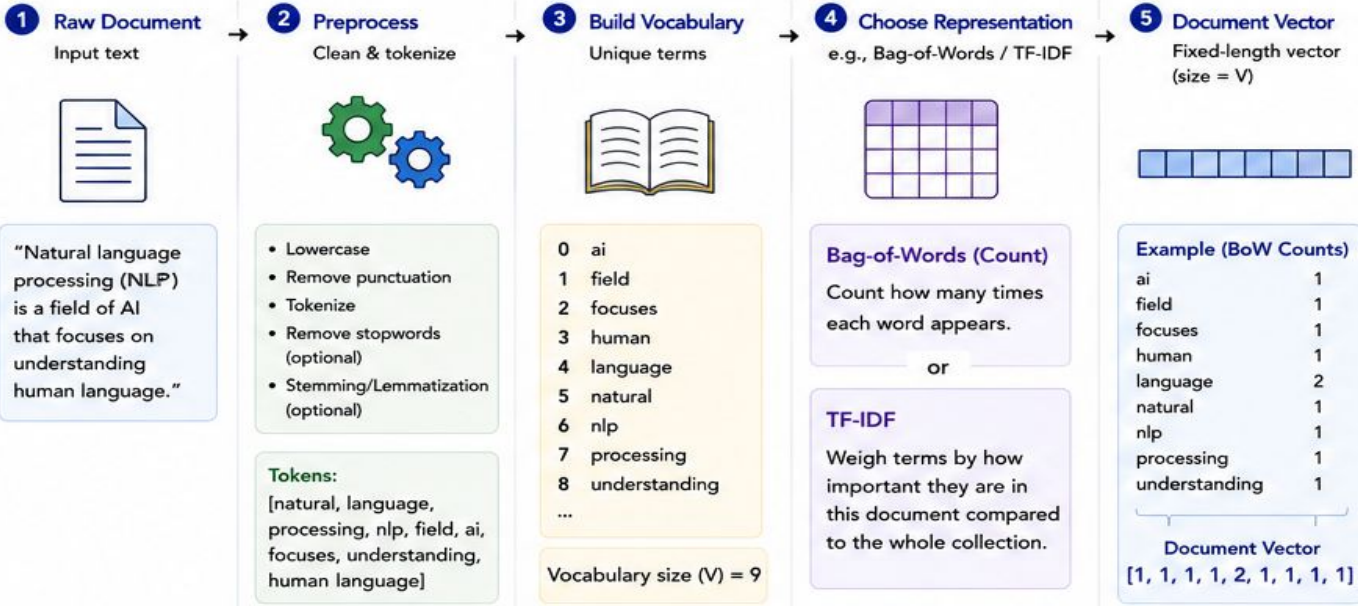
Embeddings

REPRESENTING AN ENTIRE DOCUMENT



To work with machine learning models, we convert an entire document (a variable-length text) into a **fixed-length numeric vector** that captures its content.

THE PIPELINE



COMMON DOCUMENT REPRESENTATIONS

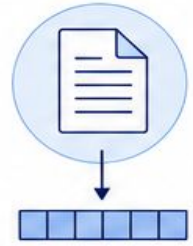
Representation	Description	Vector Size	Captures
Bag-of-Words (BoW)	Counts occurrences of each word in the document.	V (vocabulary size)	Word presence (frequency)
TF-IDF	Weights words by how important they are in this document across the whole corpus.	V	Importance of words
Averaged Word Embeddings	Average the embeddings of all words in the document.	d (embedding dimension)	Semantic meaning
Doc2Vec / Sentence Transformers	Learn a dense vector representation for the entire document.	d	Semantic meaning + context

EXAMPLE: BAG-OF-WORDS & TF-IDF

Vocabulary (V = 6)		Three Documents	Bag-of-Words (Counts)						TF-IDF (Example Values)						
0	ai	D ₁ : "AI models process data."	Doc	0	1	2	3	5	Doc	0	1	2	3	4	5
1	language		D ₁	1	0	1	1	0	D ₁	0.70	0	0.54	0.62	0.40	0
2	processing	D ₂ : "Natural language understanding."	D ₂	0	1	0	0	1	D ₂	0	0.71	0	0	0	0.71
3	models	D ₃ : "Language models for AI."	D ₃	1	1	0	1	0	D ₃	0.57	0.57	0	0.64	0	0
4	data														
5	understanding														

KEY TAKEAWAYS

- ✓ A document is **variable-length**, but we need a **fixed-length vector**.
- ✓ **BoW** is simple and intuitive but ignores word order and meaning.
- ✓ **TF-IDF** reduces the impact of common words and highlights important terms.
- ✓ Embedding-based methods capture semantics and context, producing dense and more informative vectors.



IN SHORT

We convert **text** → **tokens** → **numbers** → **vector**.
This vector is then used as input for machine learning models for tasks such as **classification**, **clustering**, **search**, and more.

WHY ONE-HOT VECTORS DO NOT CAPTURE MEANING



One-Hot Encoding represents only presence (1) or absence (0) of a category. It does not encode any information about **similarity**, **relatedness**, or **context** between categories.

1. ALL CATEGORIES ARE TREATED AS COMPLETELY INDEPENDENT

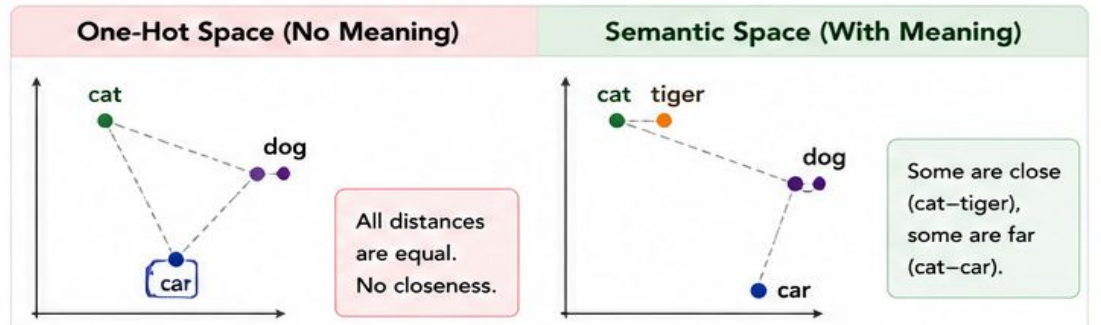
In one-hot vectors, every category is **orthogonal** (independent) to every other category.



The model sees "cat" and "tiger" as just as **unrelated** as "cat" and "car", even though cats and tigers are much more similar.

2. NO NOTION OF SIMILARITY OR DISTANCE

- ✗ One-hot vectors cannot express that some words are closer in meaning.
- ✓ There is no concept of "closeness" between vectors.



3. DOES NOT CAPTURE RELATIONSHIPS, HIERARCHIES OR CONTEXT

One-hot encoding cannot represent:

- Synonyms (e.g., big – large)
- Hierarchies (e.g., animal → mammal → cat)
- Contextual similarity (e.g., bank in "river bank" vs "money bank")

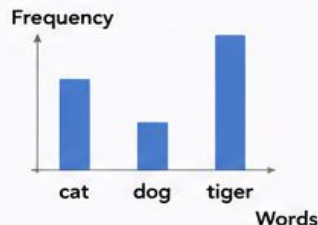
Word	animal	mammal	cat	dog	car
cat	0	0	1	0	0
dog	0	0	0	1	0
car	0	0	0	0	1

- ✗ The model cannot infer that cat and dog share a parent (mammal).
- ✗ It cannot generalize knowledge from one category to a related one.
- ✓ Meaningful representations (embeddings) are needed for this.

4. NO INFORMATION ABOUT INTENSITY OR FREQUENCY

One-hot only tells us "present" or "absent". It does not tell us how often or how important a category is.

One-hot cannot represent that "tiger" appears more frequently than "cat" or "dog".



KEY TAKEAWAY

- ✓ One-hot encoding is **simple** but **shallow**.
- ✓ It captures **identity**, not **meaning**.
- ✓ It treats all categories as **unrelated** and **equally distant**.
- ✓ To capture meaning, we need dense representations such as **Word Embeddings** or **Contextual Embeddings**.



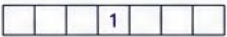
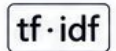
WHY WE NEED WORD EMBEDDINGS



Traditional encodings (One-Hot, Bag-of-Words, TF-IDF, etc.) represent words as symbols or counts. They **do not capture meaning, context or relationships** between words.

Word embeddings solve this by representing words as **dense vectors** of real numbers in a **continuous space**.

LIMITATIONS OF TRADITIONAL ENCODINGS

Encoding Type	What It Captures	What It Misses
One-Hot 	✓ Identity (which word)	✗ No notion of meaning ✗ All words are equally distant ✗ Very high dimensional & sparse ✗ Poor generalization
Bag-of-Words / Count 	✓ Word presence / frequency	✗ Ignores word order & context ✗ No semantic similarity ✗ High dimensional & sparse
TF-IDF 	✓ Importance of words in a document	✗ Still no meaning or relationships ✗ Vectors remain high dimensional & sparse



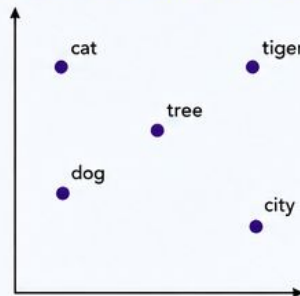
WHAT WORD EMBEDDINGS PROVIDE

	Capture Meaning	Words with similar meanings have similar vectors. king – man + woman ≈ queen
	Capture Relationships	Analogies and associations emerge naturally in the vector space.
	Compact & Dense	Low-dimensional, dense vectors (e.g., 50–300 dimensions) → efficient storage and faster computation.
	Better Generalization	Models generalize well to unseen words and new contexts.
	Improve Downstream NLP Performance	Enhances performance in tasks like classification, QA, MT, summarization, sentiment analysis, and more.

EXAMPLE: SEMANTIC NEIGHBORHOOD (2D VISUALIZATION)

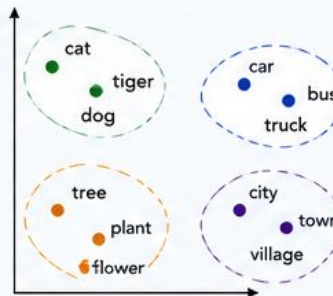
Traditional Encodings (No Meaning)

All words are equally distant and isolated.



Word Embeddings (With Meaning)

Semantically similar words are close to each other.



IN SHORT: WHY WE NEED WORD EMBEDDINGS



To teach machines the **meaning** of words, not just their identity or frequency.



To capture **relationships, similarities, and patterns** in language.



To build more **intelligent and generalizable** NLP models.



To improve performance across a **wide range** of real-world NLP applications.



KEY TAKEAWAY

Word embeddings transform words from **sparse symbols** into **dense, meaningful representations**, enabling machines to understand language **the way humans do**.

Sparse
(No Meaning)



Dense
(With Meaning)



Better NLP Models
& Applications

SPARSE vs DENSE REPRESENTATIONS



Data can be represented in many ways. Traditional methods (e.g., One-Hot, Bag-of-Words) produce **sparse vectors** with mostly zeros. Word embeddings (e.g., Word2Vec, GloVe, BERT) produce **dense vectors** with real-valued numbers that capture meaning and relationships.

1. SPARSE REPRESENTATIONS (Traditional)

High-dimensional vectors with mostly zeros.

Examples: One-Hot, Multi-Hot, Bag-of-Words, TF-IDF

Example Vocabulary (V = 8)

0 cat
1 dog
2 tiger
3 car
4 bike
5 bus
6 train
7 airplane

One-Hot Vector (V = 8)

cat → [1 0 0 0 0 0 0 0]

tiger → [0 0 1 0 0 0 0 0]

car → [0 0 0 1 0 0 0 0]

Bag-of-Words (Document: "cat tiger car")

Count Vector (V = 8)

1 0 1 1 0 0 0 0



- Very high dimensional (size = V, vocabulary size)
- Mostly zeros → very sparse
- Captures word presence or frequency only
- No notion of similarity or meaning
- High memory usage, slower computation

2. DENSE REPRESENTATIONS (Modern)

Low-dimensional vectors with real-valued numbers.

Examples: Word Embeddings (Word2Vec, GloVe, FastText, BERT)

Example: Word Embeddings (dimension = 4)

cat → [0.21 -0.35 0.57 0.02]

tiger → [0.19 -0.28 0.52 0.08]

car → [-0.31 0.44 -0.12 0.25]

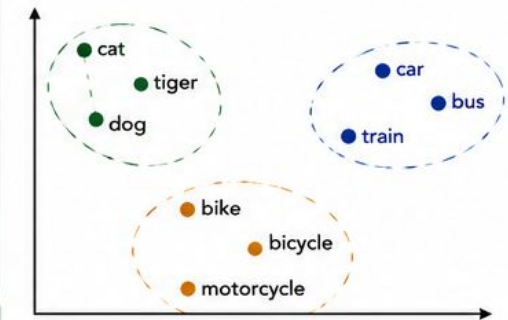
bike → [-0.22 0.41 -0.18 0.17]

bus → [-0.48 0.33 -0.21 0.36]



- Low dimensional (d << V)
- No zeros (or very few) → dense
- Captures semantic meaning & relationships
- Similar words have similar vectors
- Lower memory usage, faster computation

2D Visualization (example)



3. SPARSE vs DENSE REPRESENTATIONS – COMPARISON

Aspect	Sparse Representations	Dense Representations
Vector Size	Very high (V, can be 10K – 1M+)	Low (d, typically 50 – 500)
Number of Non-zero Values	Very few (mostly 0s)	All values are non-zero (or almost)
Data Type	Binary (0/1) or Count / TF-IDF	Real-valued numbers
Captures Meaning	✗ No	✓ Yes
Captures Similarity	✗ No (all words equally distant)	✓ Yes (similar words are close)
Memory Usage	High	Low
Computation	Slower (due to high dimension)	Faster
Example Methods	One-Hot, Multi-Hot, BoW, TF-IDF	Word2Vec, GloVe, FastText, BERT

4. WHERE EACH IS USEFUL?

Sparse Representations



- When vocabulary is small
- When we only need word presence / frequency
- Simple models (e.g., Linear models, Naïve Bayes)
- Interpretability is important

Dense Representations



- When we need to understand meaning and context
- NLP deep learning models
- Better generalization
- Tasks: translation, QA, summarization, sentiment, chatbots, etc.



KEY TAKEAWAY

Sparse vectors tell us **which words are present**.

Dense vectors tell us **what the words mean and how they relate**.

Sparse
(High-dim, Mostly 0s)



Dense
(Low-dim, Real values)



Better Understanding
& Performance

FROM ONE-HOT VECTORS TO WORD EMBEDDINGS



One-hot encoding represents words as **sparse, high-dimensional** vectors with **no notion** of meaning.
Word embeddings represent words as **dense, low-dimensional** vectors that **capture meaning** from context.

1. THE PROBLEM WITH ONE-HOT VECTORS

Vocabulary = { cat, dog, tiger, car, bike }

Vocabulary size (V) = 5

cat → [1, 0, 0, 0, 0]

dog → [0, 1, 0, 0, 0]

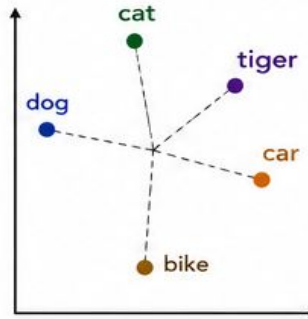
tiger → [0, 0, 1, 0, 0]

car → [0, 0, 0, 1, 0]

bike → [0, 0, 0, 0, 1]

Issues:

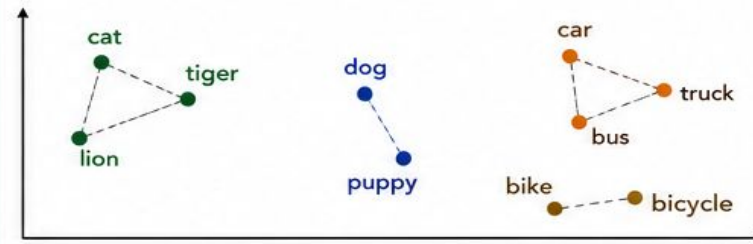
- ✗ Very high dimensional (sparse).
- ✗ All words are equally distant.
- ✗ No information about similarity or meaning.



2. THE GOAL OF WORD EMBEDDINGS

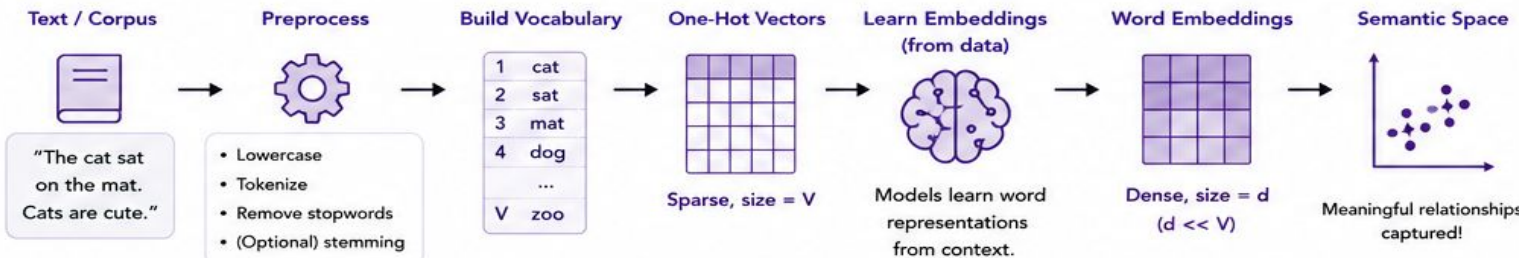
- ✓ Map each word to a **dense, low-dimensional** vector in a continuous space where **semantically similar words** are close to each other.

Example (2D visualization)



- ✓ Similar words are close.
Relationships and patterns are captured in the vector space.

3. HOW WE MOVE FROM ONE-HOT TO EMBEDDINGS



4. COMPARISON

Aspect	One-Hot Encoding	Word Embeddings
Vector Size	= Vocabulary size (V) (e.g., 10,000s)	Low dimension (d) (e.g., 50, 100, 300)
Sparsity	Very sparse (mostly 0s)	Dense (few or no 0s)
Captures Meaning	✗	✓
Similarity	All words equally distant	Similar words are close
Generalization	Poor	Good
Memory & Computation	High	Lower

5 WHY EMBEDDINGS WORK

- ★ Learned from large amounts of text.
- ★ Capture contextual usage of words.
- ★ Place semantically similar words closer together.
- ★ Enable models to understand meaning, analogies and relationships.

EXAMPLE: SEMANTIC RELATIONSHIPS CAPTURED

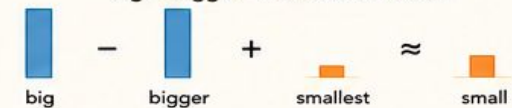
king - man + woman ≈ queen



Paris - France + Italy ≈ Rome

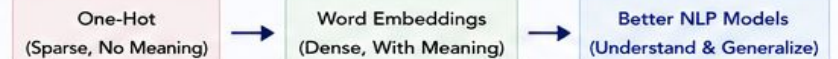


big - bigger + smallest ≈ small



KEY TAKEAWAY

One-hot vectors tell a model that words are **different**. Word embeddings tell a model how **similar, related** and **meaningful** words are.



WHAT IS WORD2VEC?



Word2Vec is a family of **unsupervised learning** models that learn **dense vector representations** (embeddings) of words from large amounts of text by capturing the **context** in which words appear.

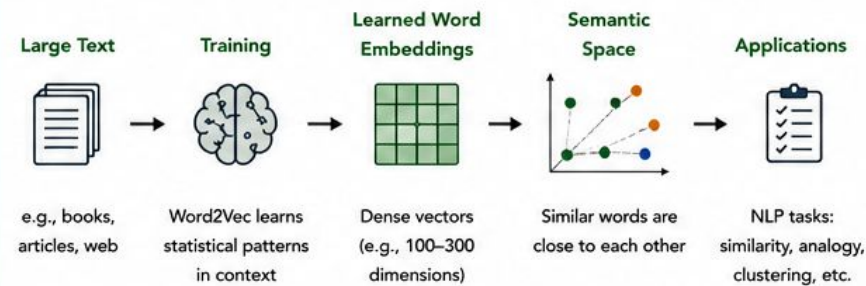
1. DEFINITION

Word2Vec maps each word to a **low-dimensional dense vector** in a **continuous space** such that **semantically similar words** have similar vectors.

Key Idea:

- Words appearing in similar contexts tend to have similar meanings.
- The model learns these patterns **automatically from data**.
- **No manual labeling** is required.

2. HOW IT WORKS – OVERVIEW

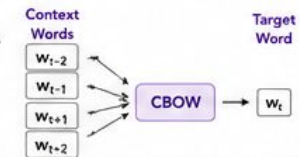


3. TWO ARCHITECTURES

Word2Vec has two main models:

• CBOW (Continuous Bag of Words)

Predicts the target word from its surrounding context.



• Skip-Gram

Predicts surrounding context words from the target word.



4. EXAMPLE – SEMANTIC RELATIONSHIPS CAPTURED

Vector Arithmetic

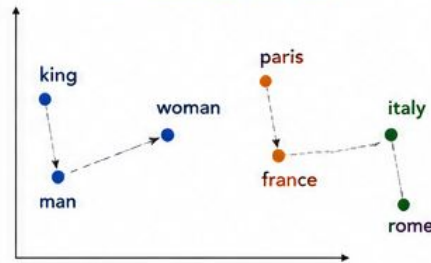
Word2Vec captures relationships such as:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

$$\text{paris} - \text{france} + \text{italy} \approx \text{rome}$$

This happens because the model learns meanings from context!

2D Visualization (example)



5. MAIN CHARACTERISTICS



Unsupervised

Learns from raw text data without labeled examples.



Dense Representations

Low-dimensional vectors (typically 50–300 dimensions).



Captures Semantics

Encodes meaning, similarity and relationships.



Efficient & Scalable

Can be trained on very large corpora.



General Purpose

Useful for many NLP tasks as a feature input.

6. COMMON APPLICATIONS



Word Similarity

Find similar words (e.g., similar, related, alike).



Analogy Tasks

Solve analogies (e.g., king : man :: queen : woman).



Text Classification

Use embeddings as features for ML/DL models.



Information Retrieval

Improve search and ranking.



Clustering & Topic Modeling

Group similar words or documents.



Machine Translation

Provide better word representations.

7. SUMMARY IN ONE LINE



Word2Vec learns word embeddings by predicting words from their context (or context from words), placing **semantically similar words** close together in a **continuous vector space**.

8. EXAMPLE OUTPUT

Example: word vectors (dimension = 4)

Word	Dim 1	Dim 2	Dim 3	Dim 4
king	-0.12	0.35	0.48	-0.05
queen	-0.10	0.32	0.47	-0.02
man	0.41	-0.23	-0.31	0.11
woman	0.39	-0.21	-0.29	0.10

Similar words (king, queen) have similar vectors.

9. WHO INTRODUCED IT?



Word2Vec was introduced by **Tomas Mikolov et al.** (Google, 2013)
Paper: *Efficient Estimation of Word Representations in Vector Space*



KEY TAKEAWAY

Word2Vec is a foundational technique in NLP that transforms words into **meaningful numeric vectors** by learning from context, enabling machines to understand **similarity, relationships** and **meaning**.



HOW WORD2VEC LEARNS WORD REPRESENTATIONS



Word2Vec learns **dense vector representations (embeddings)** of words by analyzing the **context** in which words appear. It uses the idea: Words that occur in **similar contexts** tend to have **similar meanings** (Distributional Hypothesis).

1. INPUT: LARGE TEXT CORPUS

Word2Vec is trained on a large collection of text.

Example sentences:

- The cat sat on the mat.
- The dog chased the cat.
- The cat drank some milk.
- The dog drank water.

...



2. CREATE CONTEXT-WINDOWS

For each target word, Word2Vec looks at the surrounding words within a window of size "c".

Example (window size = 2)

Sentence	Target Word	Context Words
The cat sat on the mat	cat	[the, sat, on, the]
The dog chased the cat	dog	[the, chased, the, cat]
The cat drank some milk	drank	[cat, some, milk]
...	...	...

Model learns from thousands/millions of such (target, context) pairs.

3. LEARNING OBJECTIVE

Word2Vec learns to predict context words from a target word (Skip-Gram) or the target word from context words (CBOW).

In both cases, the model learns patterns of co-occurrence from data.

Goal: Maximizing the probability of correct word predictions.

Mathematically (Skip-Gram example):

$$\max_{\theta} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

where w_t = target word, w_{t+j} = context word

4. RESULT: WORD VECTORS

After training, each word is represented by a d-dimensional dense vector.

Example: 4-dimensional vectors

Word	Vector (d = 4)
cat	[0.21 -0.35 0.57 0.02]
dog	[0.19 -0.31 0.54 0.05]
king	[0.62 0.11 0.09 0.73]
queen	[0.58 0.13 0.08 0.70]
milk	[0.12 -0.42 0.43 0.33]
...	...



Words with **similar meanings** or used in **similar contexts** have **similar vectors**.

5. TRAINING PROCESS (HIGH-LEVEL FLOW)



1. Corpus
Input text collection



2. Preprocess
Tokenize, lowercase,
(remove stopwords
optional)



3. Generate (target, context) pairs
Slide window across sentences



4. Train Word2Vec Model
Using CBOW or Skip-Gram
(optimization with
Stochastic Gradient Descent)



5. Learn Word Vectors
Each word gets a dense
vector of dimension d



6. Semantic Space
Words are placed in a
2D continuous space
where similar words are close together

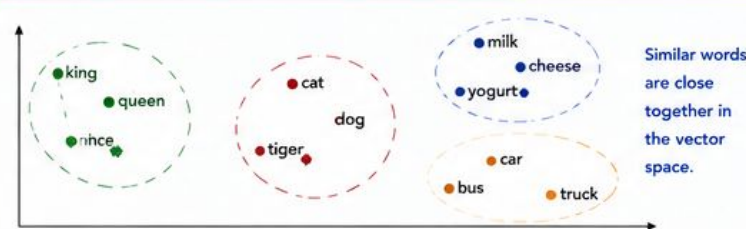
6. KEY TAKEAWAYS

- ✓ Learns from raw text automatically (no manual labeling).
- ✓ Captures semantics and relationships between words.
- ✓ Produces low-dimensional dense vectors (e.g., 50–300 dims).
- ✓ Enables better generalization in NLP tasks.
- ✓ Foundation for many modern language models.

7. EXAMPLES OF CAPTURED RELATIONSHIPS

Relationship Type	Example	What It Captures
Synonymy	big – large, small – tiny	Similar meaning
Analogy	king – man + woman ≈ queen	Relationship & patterns
Category	cat – dog – tiger	Belong to similar category
Function	knife – cut, pencil – write	Functional similarity

8. 2D VISUALIZATION EXAMPLE (t-SNE/PCA)



IN SHORT



Word2Vec looks at how words appear together in text.



It learns patterns by predicting words using a neural network.



It produces dense vectors where meaningful words are close together.



KEY MESSAGE

Word2Vec learns word representations by leveraging **context**.

It transforms words into **meaningful numeric vectors** that machines can understand and use.

TWO WORD2VEC ARCHITECTURES – CBOW & SKIP-GRAM



Word2Vec learns word embeddings using two complementary architectures:

CBOW (Continuous Bag of Words) predicts a **target word** from its context.

Skip-Gram predicts context words from a target word.

1. CBOW (CONTINUOUS BAG OF WORDS)

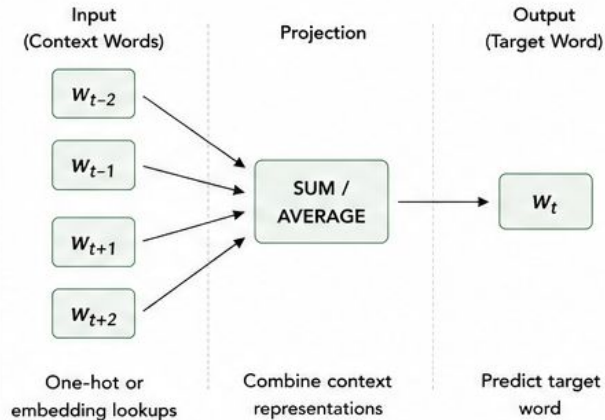
Goal: Given the surrounding context words, predict the target word.

Idea

Use many context words to predict the center word.

Good at capturing:

- ✓ General semantics
- ✓ Frequent words
- ✓ Smaller datasets



2. SKIP-GRAM

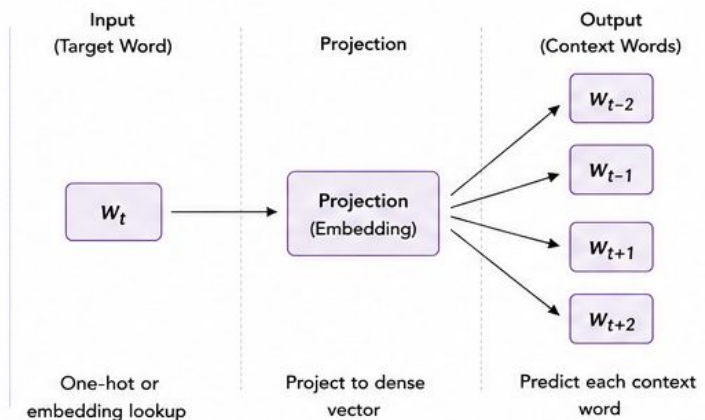
Goal: Given the target word, predict the surrounding context words.

Idea

Use the center word to predict multiple context words.

Good at capturing:

- ✓ Rare words
- ✓ Specific meanings
- ✓ Larger datasets



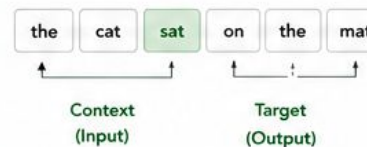
3. SIDE-BY-SIDE COMPARISON

Aspect	CBOW	Skip-Gram
Direction	Context → Target	Target → Context
Input	Multiple context words	Single target word
Output	Single target word	Multiple context words
Best For	Frequent words, general meaning	Rare words, specific meaning
Training Speed	Faster	Slower
Data Efficiency	Works well with less data	Often needs more data
Typical Use	Classification, general tasks	Information retrieval, semantics

4. EXAMPLE (Window size = 2)

CBOW Example

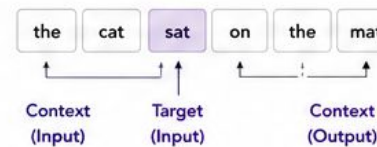
Sentence: "the cat sat on the mat"



Model learns: [the, cat, on, the] → sat

Skip-Gram Example

Sentence: "the cat sat on the mat"



Model learns: sat → [the, cat, on, the]

5. WHEN TO USE WHAT?

Use CBOW when:

- ✓ Dataset is small or medium
- ✓ Speed is important
- ✓ You care more about frequent words and general meaning

Use Skip-Gram when:

- ✓ Dataset is large
- ✓ You want better performance on rare words
- ✓ You care more about semantic relationships



KEY TAKEAWAY

CBOW and Skip-Gram learn the same type of word embeddings but from different directions. Together, they capture rich semantic information from text.



Outcome: Dense vector representations where semantically similar words are close to each other.

CBOW — WORKED EXAMPLE



CBOW (Continuous Bag of Words) predicts the **target (center) word** from its surrounding **context words**.
It uses many context words to predict **one target word**.

1. THE SENTENCE & WINDOW

Sentence:

"The cat sat on the mat"

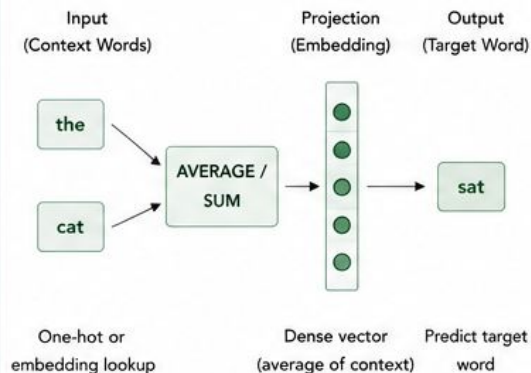
Window size = 2

(We use 2 words on each side)

Position	Context Words	Target Word
1	-	-
2	The	cat
3	The, cat	sat
4	cat, sat	on
5	sat, on	the
6	on, the	mat
7	-	-

Let's work on Position 3 → target word = sat
Context words = [the, cat]

2. CBOW ARCHITECTURE (Overview)



3. NUMERICAL EXAMPLE (Using Embeddings)

Assume each word is represented by a 4-dimensional vector.
(These are **example vectors**)

Word	Embedding (dimension = 4)			
	d_1	d_2	d_3	d_4
the	0.22	-0.10	0.41	0.05
cat	0.31	0.20	-0.13	0.07
sat (target)	-0.25	0.08	0.19	-0.22

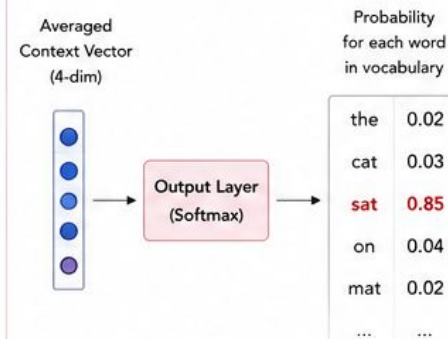
Step 1: Average the context word vectors

Context = [the, cat]

$$v_{\text{context}} = \frac{v_{\text{the}} + v_{\text{cat}}}{2}$$
$$= \left[\frac{0.22+0.31}{2}, \frac{-0.10+0.20}{2}, \frac{0.41-0.13}{2}, \frac{0.05+0.07}{2} \right]$$
$$= [0.265, 0.050, 0.140, 0.060]$$

4. PREDICTION (Output Layer)

The averaged vector is used to predict the target word "sat".



The model assigns the highest probability to "sat", which is our target word.

5. TRAINING OBJECTIVE

The model learns by maximizing the probability of the correct target word.

For this example (Skip-Gram objective form shown for clarity):

$$\max_{\theta} \log P(\text{sat} | \text{the, cat})$$

The model adjusts the weights so that the probability of the correct target word (sat) becomes higher.

Key Points

- ✓ Uses many context words to predict one target word.
- ✓ Works well for frequent words and smaller datasets.
- ✓ Faster to train.

6. ANOTHER POSITION (Quick Look)

Position 4 → target word = on

Context = [cat, sat]

• $v_{\text{cat}} = [0.31, 0.20, -0.13, 0.07]$

• $v_{\text{sat}} = [-0.25, 0.08, 0.19, -0.22]$

Average:

$$v_{\text{context}} = \frac{v_{\text{cat}} + v_{\text{sat}}}{2}$$
$$= \left[\frac{0.31-0.25}{2}, \frac{0.20+0.08}{2}, \frac{-0.13+0.19}{2}, \frac{0.07-0.22}{2} \right]$$
$$= [0.03, 0.14, 0.03, -0.075]$$

Model predicts → on

7. SUMMARY



CBOW takes surrounding words (context) as input.



It averages (or sums) their vectors to get a single context representation.



This representation is used to predict the center (target) word.



Through training on millions of such examples, good word vectors are learned.



KEY TAKEAWAY

CBOW = Context → Target.

It predicts the center word from its surroundings by learning **dense vector representations** of words.



Better at: frequent words, efficiency, speed.

SKIP-GRAM — WORKED EXAMPLE



Skip-Gram predicts the surrounding context words from a **target (center) word**.
It uses one center word to predict multiple **context words**.

1. THE SENTENCE & WINDOW

Sentence:

"The cat sat on the mat"

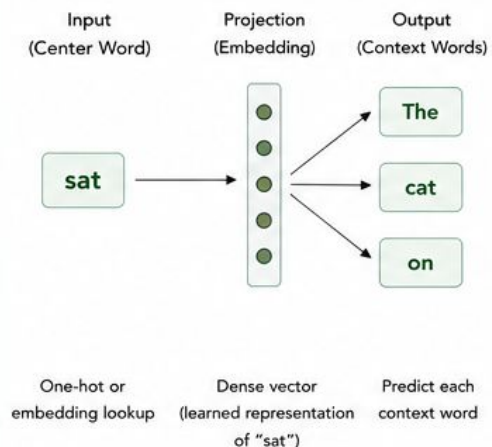
Window size = 2

(We use 2 words on each side)

Position	Words	Center Word	Context Words
1	The	-	-
2	cat	cat	The, sat
3	sat	sat	The, cat, on
4	on	on	cat, sat, the, mat
5	the	the	sat, on, mat
6	mat	mat	on, the

Let's work on Position 3 → center word = **sat**
Context words = [The, cat, on]

2. SKIP-GRAM ARCHITECTURE (Overview)



3. NUMERICAL EXAMPLE (Using Embeddings)

Assume each word is represented by a 4-dimensional vector. (These are **example vectors**)

Word	Embedding (dimension = 4)			
	d_1	d_2	d_3	d_4
The	0.22	-0.10	0.41	0.05
sat (center)	-0.25	0.08	0.19	-0.22
cat	0.31	0.20	-0.13	0.07
on	0.18	-0.05	0.23	-0.10
mat	-0.12	0.30	-0.07	0.04

For center word "sat", the model learns to predict each context word separately.

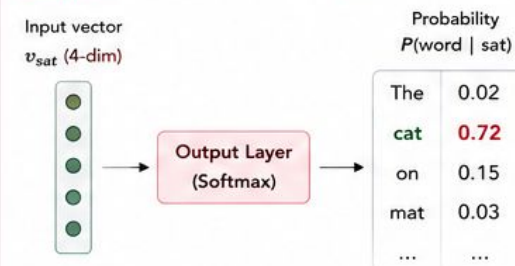
Input vector (sat):

$v_{sat} = [-0.25, 0.08, 0.19, -0.22]$

4. PREDICTION (Output Layers)

From the input vector of "sat", the model predicts each context word using an output layer (softmax).

Example: Predicting context word "cat"



The model assigns the highest probability to "cat", which is the correct context word.

5. TRAINING OBJECTIVE

Skip-Gram maximizes the probability of all context words given the center word.

For center word c and its context words $w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$:

$$\max_{\theta} \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

In our example (window=2, center="sat"):

Maximize $\log P(\text{The}|\text{sat}) + \log P(\text{cat}|\text{sat}) + \log P(\text{on}|\text{sat})$

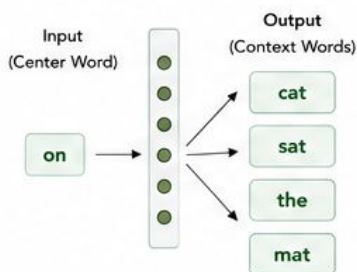
The model updates its weights so that the probabilities of the correct context words become higher.

Key Points

- ✓ Uses one center word to predict many contexts.
- ✓ Better at capturing rare words and specific meanings.
- ✓ Usually needs more data and is slower to train.

6. ANOTHER POSITION (Quick Look)

Position 4 → center word = **on**
Context words = [cat, sat, the, mat]



The model learns to predict four different context words from the same center word.

Each (center, context) pair is a training example.

7. SUMMARY



Skip-Gram = Target → Context.



Uses one center word to predict all its surrounding context words.



Captures rare words and fine-grained meanings better than CBOW.



Typically requires more training data and computational time.



Produces dense vector representations where words with similar contexts are close together in the vector space.



KEY TAKEAWAY

CBOW predicts the center word from its context (**Context → Target**),
while Skip-Gram predicts the context words from the center word (**Target → Context**).



Both architectures learn meaningful word embeddings that power many NLP applications.

LIMITATIONS OF WORD2VEC



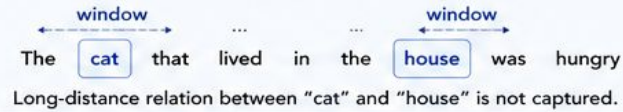
Word2Vec is powerful and simple, but it also has **important limitations** that we should be aware of.

1 Context is Limited to Local Window

Word2Vec only looks at a small window of surrounding words.

- It may miss long-range dependencies.

Sentence: The **cat** that lived in the **house** was hungry.



2 Ignores Word Order in CBOW

CBOW uses a bag (average) of context words, ignoring their order.

- Different sentences with same words but different order look identical.

The **dog** chased the **cat**.
The **cat** chased the **dog**.

→ Same context bag for "chased"
{ the, **dog**, **cat** }
(Order lost)

3 No Explicit Handling of Polysemy

A word with multiple meanings gets only one vector.

- Different senses are mixed together.

"bank" (financial institution)
vs.
"bank" (river side)

Same vector representation (one point in space)

4 Cannot Handle Out-of-Vocabulary (OOV) Words

New words not seen during training have no vector.

- Need to retrain or use special tricks (e.g., subword models).

Training vocabulary
cat, dog, house, run, eat, big, ...

✗ Unknown word: "giraffe"
No vector available!

5 Static Embeddings

Word2Vec gives the same vector regardless of context.

- It cannot capture how a word's meaning changes in different situations.

"apple" in a sentence about fruit
vs.
"apple" in a sentence about technology

Same vector for both contexts

6 Limited to Linear Relationships

Word2Vec captures some linear relationships (e.g., king - man + woman ≈ queen) but struggles with complex, non-linear ones.



KEY TAKEAWAY

Word2Vec is:

- ✓ Fast
- ✓ Simple
- ✓ Effective with limited data
- ✓ Great as a baseline

But it:

- ✗ Looks only at local context
- ✗ Ignores order (in CBOW)
- ✗ Does not handle multiple meanings
- ✗ Cannot represent unseen words
- ✗ Produces static representations
- ✗ Captures only limited (linear) relationships



These limitations motivated the development of contextual embeddings (e.g., ELMo, BERT, GPT) that address many of these issues.



HOW MODERN MODELS ADDRESS THESE

- ✓ Use larger context (Transformers, attention)
- ✓ Capture word order and position (Transformers)

Provide contextual (dynamic) embeddings (BERT, GPT)

Handle OOV with subword/tokenization (BPE, WordPiece)

Learn complex, non-linear relationships (Deep models)